

Here is the output of these programs:

---

```
racecar is a palindrome: True
amanaplanacanalpanama is a palindrome: True
tacocat is a palindrome: True
zophie is a palindrome: False
```

---

The base case returns `True` because a zero- or one-character string is always a palindrome. Otherwise, the string argument is broken into three pieces: the first character ❶, the last character ❸, and the middle characters between them ❷.

The `return` statement in the recursive case ❹ makes use of *Boolean short-circuiting*, a feature of almost every programming language. In an expression joined with the `and` or `&&` Boolean operators, if the left-side expression is `False`, it doesn't matter if the right-side expression is `True` or `False` because the entire expression will be `False`. Boolean short-circuiting is an optimization that skips the evaluation of the right-side expression of an `and` operator if the left side is `False`. So, in the expression `head == last and isPalindrome(middle)`, if `head == last` is `False`, the recursive call to `isPalindrome()` is skipped. This means that as soon as the head and last strings don't match, the recursion stops and simply returns `False`.

This recursive algorithm is still sequential, like the summation and reverse-string functions in the previous sections, except that instead of going from the start of the data to the end, it goes from both ends of the data toward the middle. The iterative version of this algorithm that uses a simple loop is more straightforward. We cover the recursive version in this book because it's a common coding interview problem.